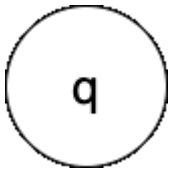
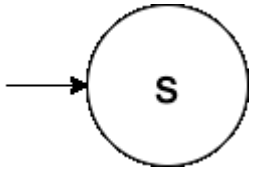
Automates à états finis

Plan

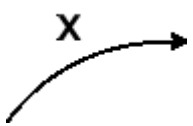
1. Automates finis
2. Fonctionnement d'un automate fini
3. Rendre déterministe un AFN
4. Minimisation d’un AFD
5. Langage accepté par un automate fini
6. Automates finis et langages réguliers
7. Automates avec ε-transitions

Automates finis

- Un automate fini correspond à un graphe orienté, dans lequel certains des nœuds (états) sont distingués et marqués comme initial ou finaux et dans lequel les arcs (transitions) sont étiquetés par des symboles de Σ.
  - état 

  - état initial 

  - état final (ou d'acceptation) 

  - transition sur le symbole x 

Automates finis

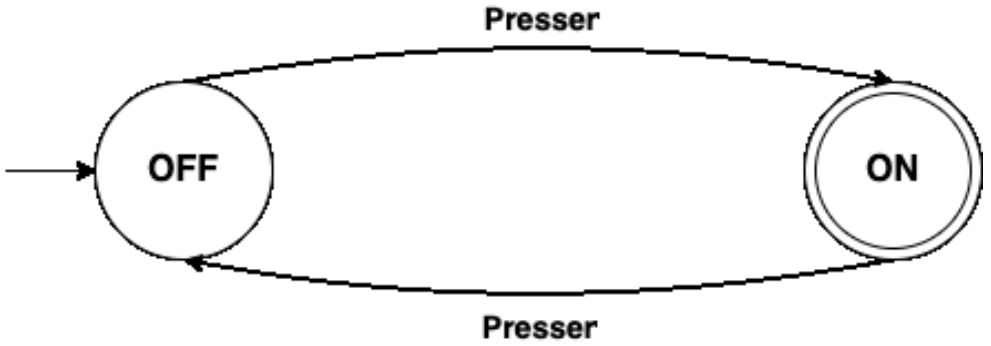
Définition :

Un automate fini consiste en un modèle de la forme (Q, Σ, δ, S, F) avec :

- **Q** est un ensemble fini d'états de l'automate
- **Σ** est un alphabet
- **δ** est une fonction de transition qui fait correspondre à des couples (état × symbole) des états
  - $\delta: Q \times \Sigma \rightarrow P(Q)$
  - (q,a) =p: transition de q à p en lisant (reconnaissant) le symbole a
- **S** est l'état initial
  - $S \in Q$
- **F** est l'ensemble des états finaux (ou états d'acceptation)
  - $F \subseteq Q$

Automates finis

Exemple 1 :

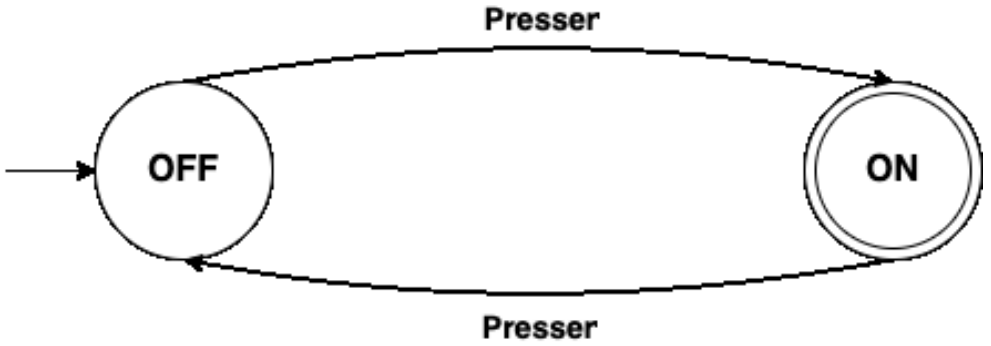


- La séquence: **Presser Presser Presser** est acceptée par l'automate
  - en partant de l'état off, avec un "Presser" on passe à l'état ON
  - En lisant le deuxième "Presser" à partir de l'état ON, on revient à l'état OFF
  - Le dernier "Presser" lu nous mène de l'état OFF à l'état ON (état final)

➠ Le mot est lu (trois Presser)et l'état est final (l'état ON) donc le mot est accepté.

Automates finis

Exemple 1 :

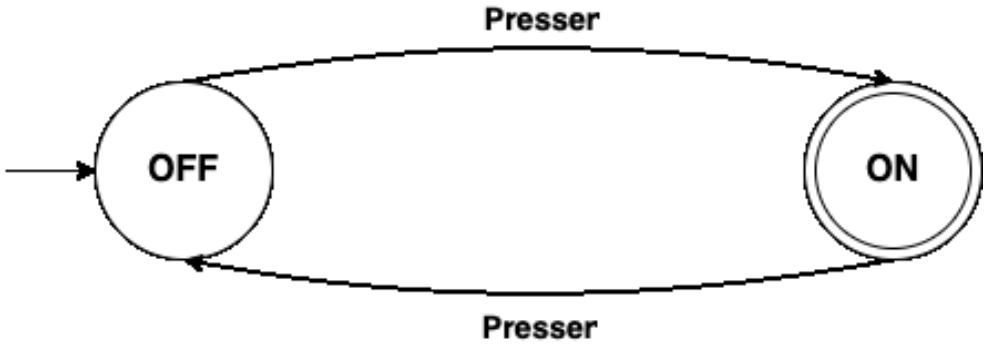


- La séquence: **Presser Presser** n'est pas acceptée par l'automate
  - en partant de l'état OFF, avec un Presser on passe à l'état ON
  - en lisant le deuxième Presser à partir de l'état ON, on revient à l'état OFF

Automates finis

Exemple 1 :

Presser(Presser Presser)*



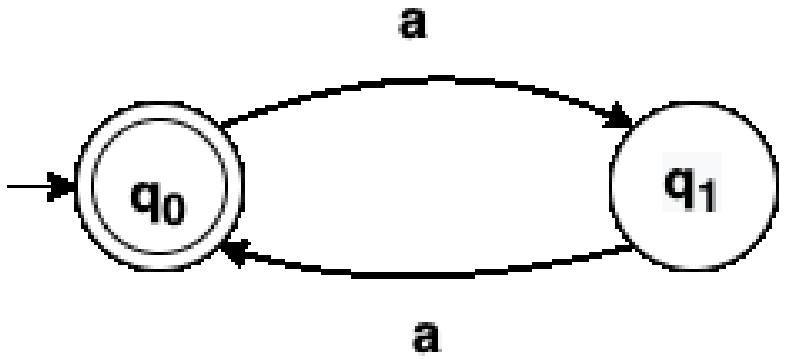
- Le langage accepté par cet automate A est l'ensemble des séquences de Presser qui, partant de l'état initial OFF, après la lecture de tous les symboles de la séquence, on se trouve à l'état ON.
- Ce langage, noté L(A), est l'ensemble des séquences de Presser de longueur impaire.

$$L(A) = \{\omega \in \{Presser\}^* / \ |\omega| = 2k + 1, k \geq 0\}$$

Automates finis

Exemple 2 :

(aa)*



- Exemple d'un automate fini qui accepte $\{\omega \in \{a\}^* / \ |\omega| = 2k, k \geq 0\}$
- q_0 est à la fois état initial et final puisque le mot ε est accepté (pour k = 0)

Automates finis

Exercice :

1) Construire l'automate $A = (Q, \Sigma, \delta, S, F)$

- $Q = \{a,b,c,d,e\}$
- $\Sigma=\{0,1\}$
- $S=\{a\}$
- $F = \{e\}$

δ	**0**	**1**
a	a,b,c,d,e	d,e
b	c	e
c	-	b
d	e	-
e	-	-

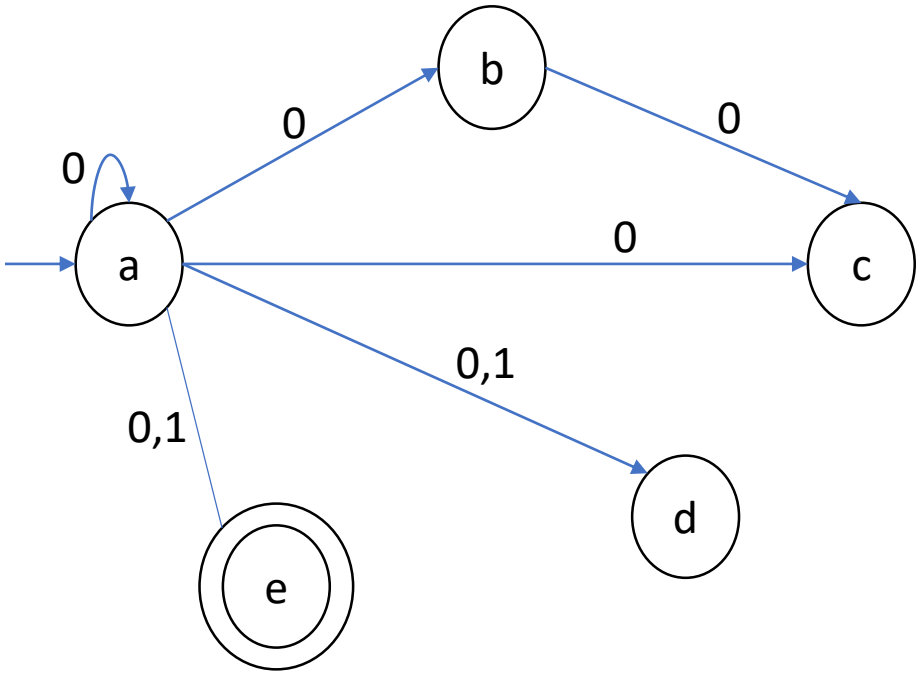
2) Construire un automate fini qui acceptent les mots du langage :

$$L = \{\omega \in \{a\}^* / \ |\omega| = 2k, k > 0\}$$

Automates finis

1) Construire l'automate A = (Q, Σ, δ, S, F)

- Q = {a,b,c,d,e} ensemble des etats
- Σ={0,1} alphabet
- S = {a} etat initial
- F = {e} etat final

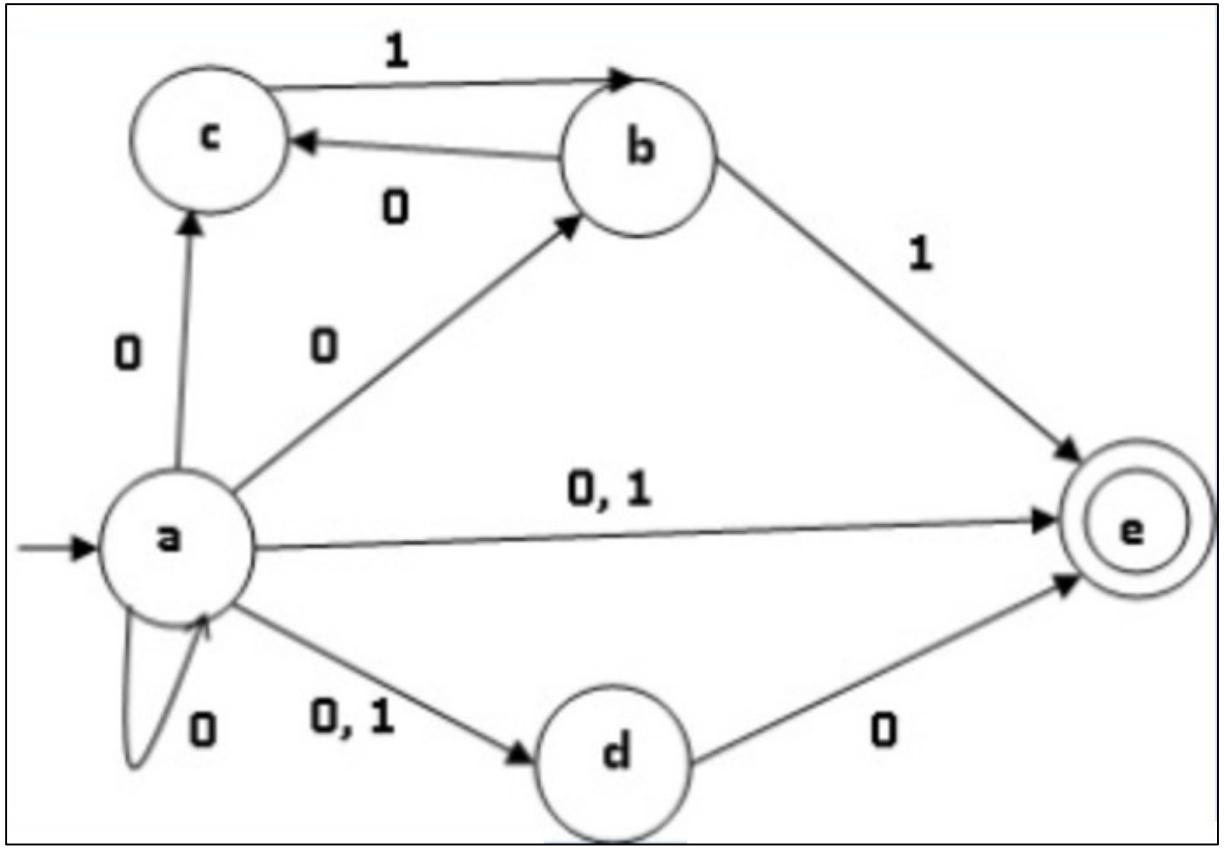


		symboles	
	δ	**0**	**1**
etats	**a**	a,b,c,d,e	d,e
	b	c	e
	c	-	b
	d	e	-
	e	-	-

Automates finis

Correction :

δ	0	1
a	a,b,c,d,e	d,e
b	c	e
c	-	b
d	e	-
e	-	-

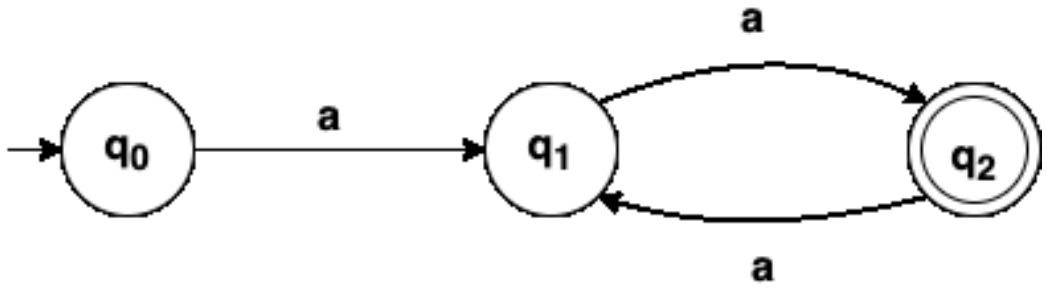


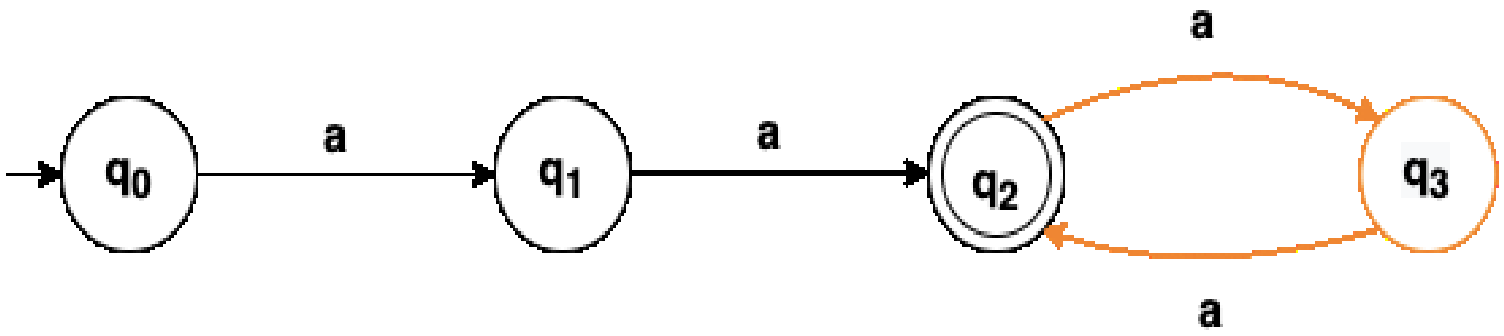
Automates finis

Correction :

2) $L = \{\omega \in \{a\}^* / \ |\omega| = 2k, k > 0\}$

aa(aa)*

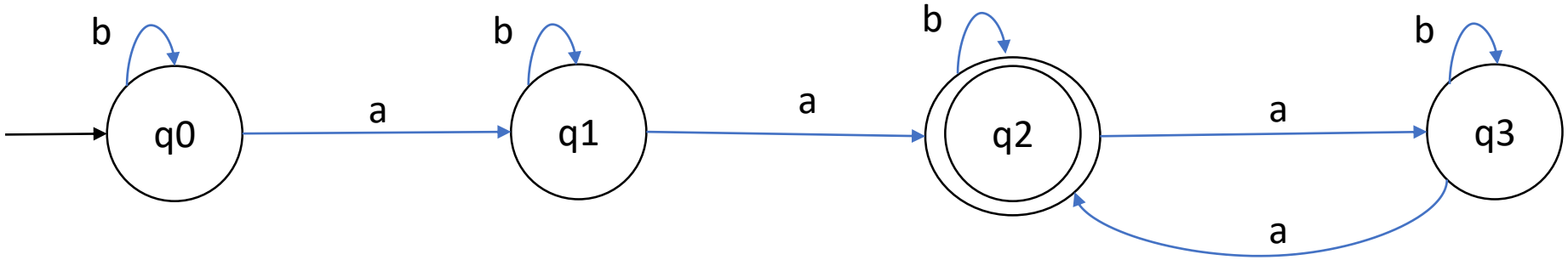


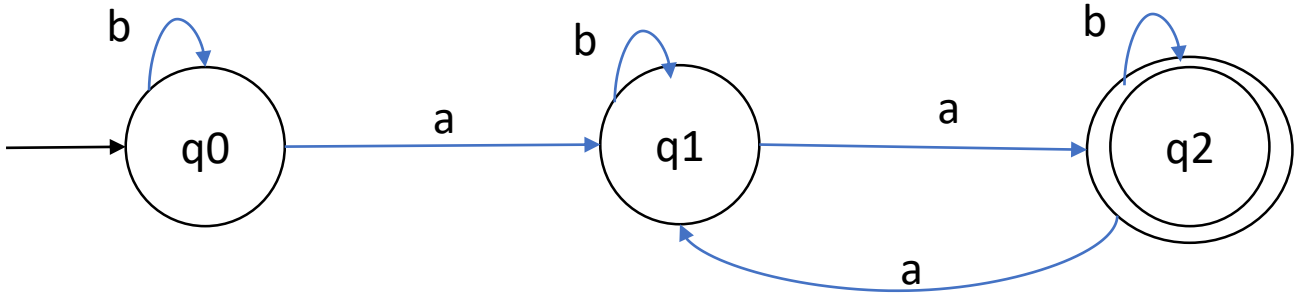


Construire un automate fini qui acceptent les mots du langage :

$$L = \{\omega \in \{a,b\}^* / \ |\omega|_a = 2k,\ k > 0\}$$

un nombre pair de a >=2





Automates finis

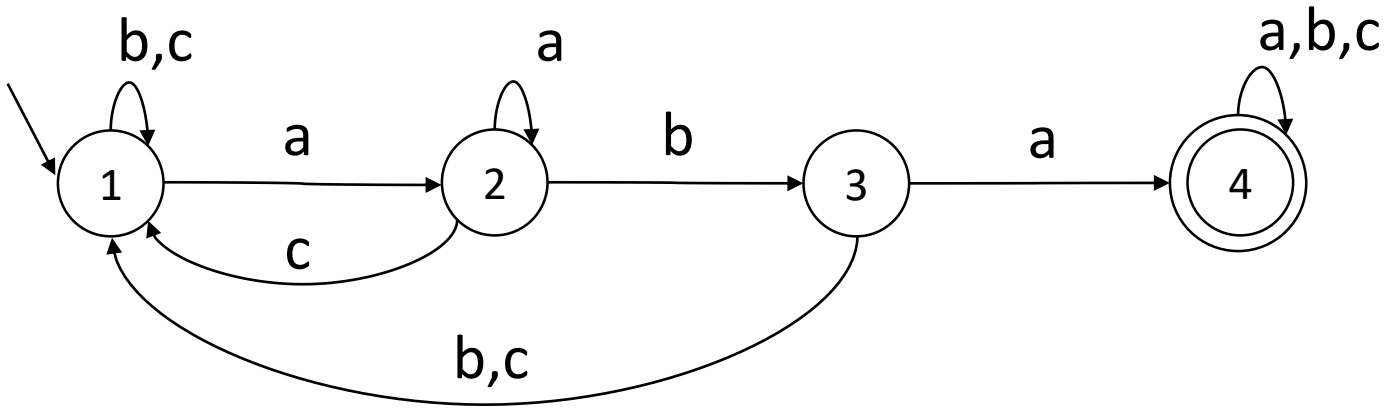
Exercice :

- Construire un automate fini qui accepte le langage suivant :
  - $L = \{w \in \{\mathrm{a}, b, c\}^* \text{ ayant aba comme facteur}\}$:
    les séquences $(\mathrm{a} + \mathrm{b} + \mathrm{c})^* \mathrm{aba}\ (\mathrm{a} + \mathrm{b} + \mathrm{c})^*$

Automates finis

Exercice 1 : Solution

- $(a + b + c)^* aba\ (a + b + c)^*$



Fonctionnement d'un automate fini

Mot connu par un automate :

- Formellement, on définit la fonction $\delta^*(q, \omega)$ qui nous dit dans quel état on arrive en partant de l'état q et en analysant le mot ω.
  - si ω est le mot vide $\delta^*(q,\omega)=q$
  - si ω commence par le symbole s suivi du sous-mot u:

    $\delta^*(q,su)=\delta^*(\delta(q,s),u)$
- Un mot ω est accepté par l'automate A dont l'état initial est q_0 si

  $\delta^*(q_0,\omega)$ appartient aux états finaux de A.

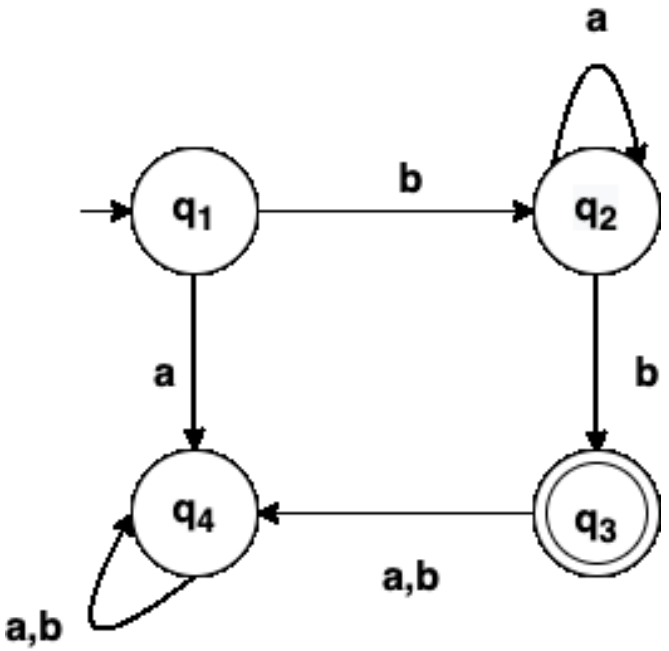
Fonctionnement d'un automate fini

Exemple :

soient

- Σ = {a,b}
- A = (Q, Σ, δ, S, F)
  - Q={q1, q2, q3, q4}
  - S={q1}
  - F={q3}

δ	a	b
q1	q4	q2
q2	q2	q3
q3	q4	q4
q4	q4	q4



Est-ce que les mots **ε**, **baab** et **abab** sont reconnus par l'automate A?

Fonctionnement d'un automate fini

Exemple :

$\delta^*(q1, \varepsilon)=q1 \notin F$ donc ε n'est pas reconnu par A

$\delta^*(q1,baab) = \delta^*(\delta(q1, b),aab)$

$= \delta^*(q2,aab)= \delta^*(\delta(q2,a),ab)$

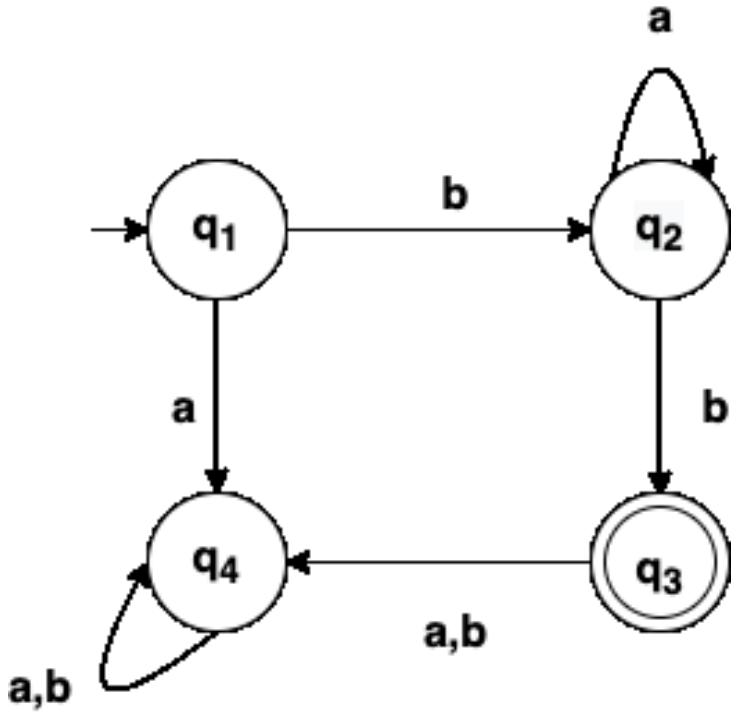
$= \delta^*(q2,ab)= \delta^*(\delta(q2,a), b)$

$= \delta^*(q2, b) = \delta^*(\delta(q2,b), \varepsilon)$

$= \delta^*(q3,\varepsilon)$

$=q3 \in F$

donc baab est reconnu par A.



Fonctionnement d'un automate fini

Exemple :

$\delta^*(q1, abab)=\delta^*(\delta(q1, a), bab)$

$= \delta^*(q4, bab)$

$= \delta^*(\delta(q4,b),ab)$

$= \delta^*(q4,ab)$

$= \delta^*(\delta(q4,a), b)$

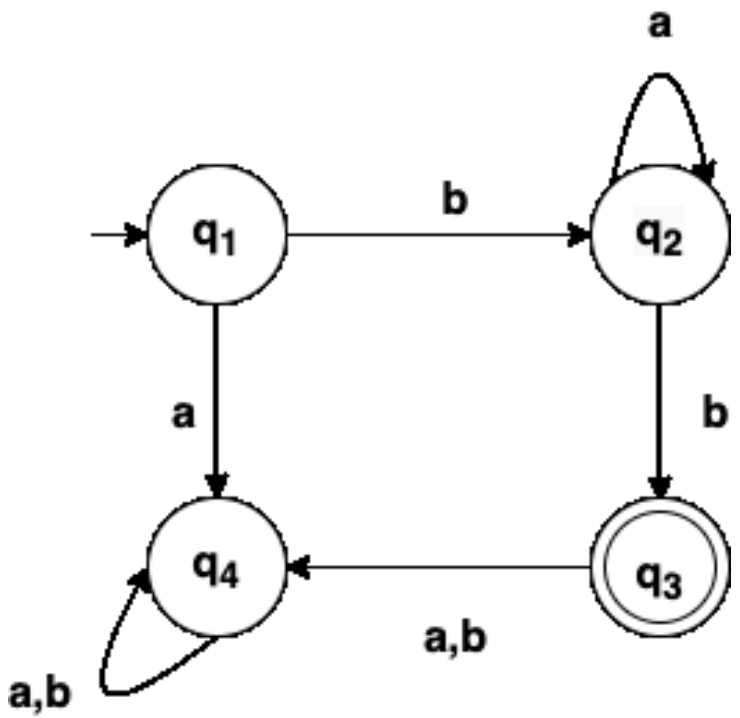
$= \delta^*(q4, b)$

$= \delta^*(\delta(q4,b),\varepsilon)$

$= \delta^*(q4,\varepsilon)$

$= q4 \notin F$

donc abab n'est pas reconnu par A



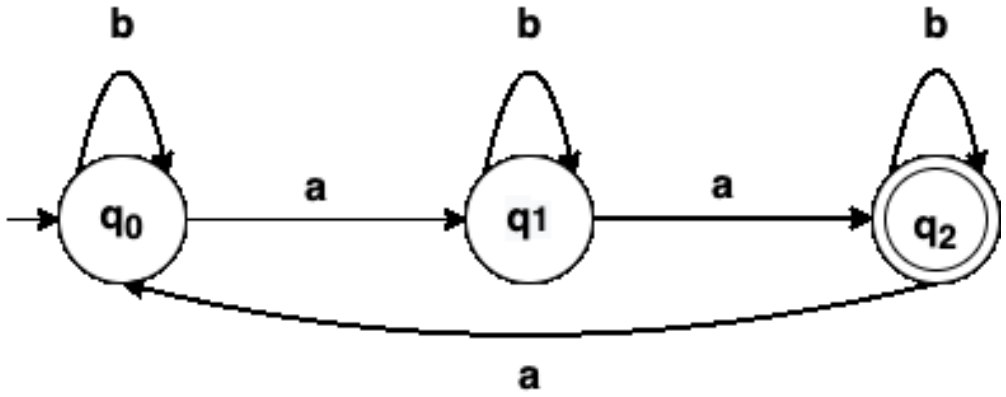
Fonctionnement d'un automate fini

Configuration :

Une configuration est un couple (q, ω) où

- q est l'état courant
- ω est le reste du mot à lire

Soit l'automate suivant:



Exemples de configurations possibles :

- (q0, aa) (q1, a) (q2, ε)
- (q0, bba) (q0, ba) (q0, a) (q1, ε)

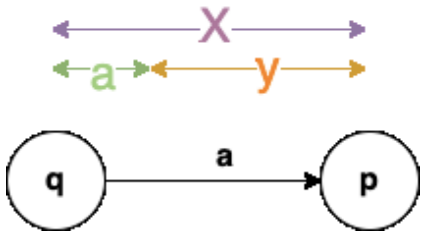
Fonctionnement d'un automate fini

Configuration successeur :

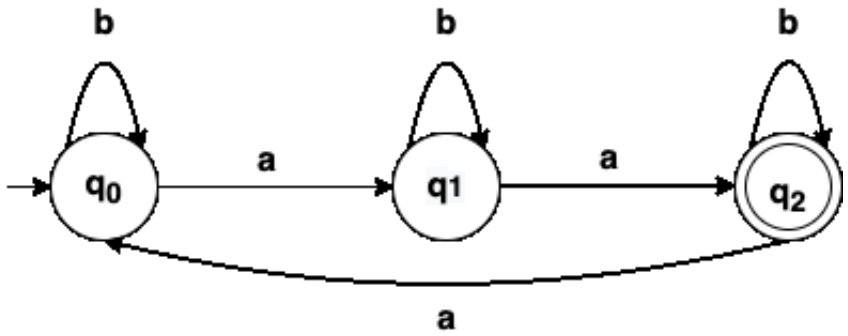
Une configuration (p, y) est successeur d'une configuration (q, x) qu'on note:

(q, x) ⊢ (p, y)

ssi ∃a / x=ay et δ(q,a)={p}



Exemple :



(q0, aa) ⊢ (q1, a) ⊢ (q2, ε)

(q0, bba) ⊢ (q0, ba) ⊢ (q0, a) ⊢ (q1, ε)

Fonctionnement d'un automate fini

Configuration kème successeur :

Une configuration (p, y) est kème successeur d'une configuration (q, x) qu'on note:

$(q, x) \vdash^{k} (p, y)$

ssi

$(q, x) \vdash (q_1, x_1) \vdash (q_2, x_2) \vdash \ldots \vdash (q_k, x_k) = (p, y)$

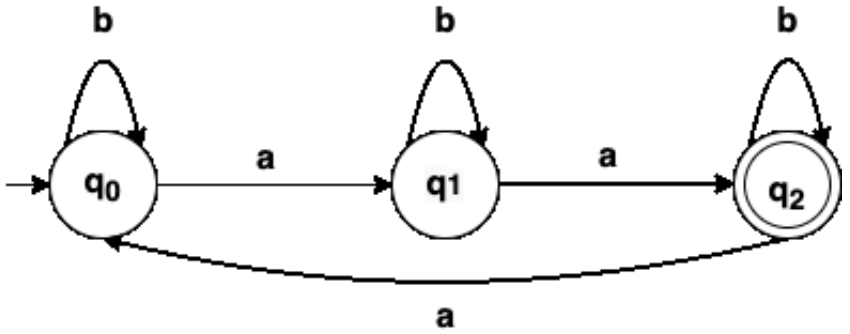
configuration successive d'un nombre quelconque d'états de transition

$(q,x) \vdash^{*} (p,y)$ ssi $\exists\ k \geq 0$ / $(q,x) \vdash^{k} (p,y)$

Fonctionnement d'un automate fini

Exemple

Pour l'automate suivant, nous avons :



$(q0,aa) \overset{*}{\vdash} (q2,\varepsilon)$ puisque: $\exists\ k(=2)\geq 0$ / $(q0,aa) \overset{k}{\vdash} (q2,\varepsilon)$

$(q0, aa) \vdash (q1, a) \vdash (q2, \varepsilon)$

$(q0, bba) \overset{3}{\vdash} (q1, \varepsilon)$ car on a:

$(q0, bba) \vdash (q0, ba) \vdash (q0, a) \vdash (q1, \varepsilon)$

Automates finis déterministes

Automate fini complet

Un automate fini est dit complet sur un alphabet Σ ssi pour chaque état q et chaque symbole s, il existe au moins une transition qui quitte q avec le symbole s.

Automate fini non ambigu

Un automate fini est dit non ambigu sur un alphabet Σ ssi pour chaque état q et chaque symbole s, il existe au plus une transition qui quitte q avec le symbole s.

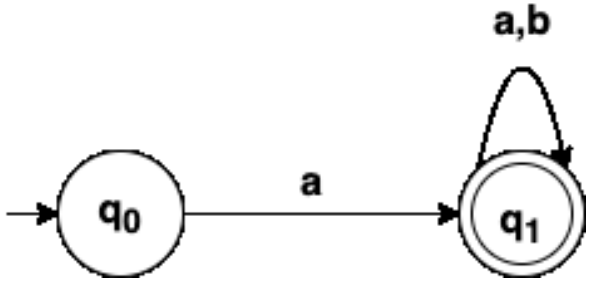
Automate fini déterministe

Un automate fini est dit déterministe (AFD) sur un alphabet Σ ssi il est complet et non ambigu (pour chaque état q et chaque symbole s, il existe une et une seule transition qui quitte q avec le symbole s).

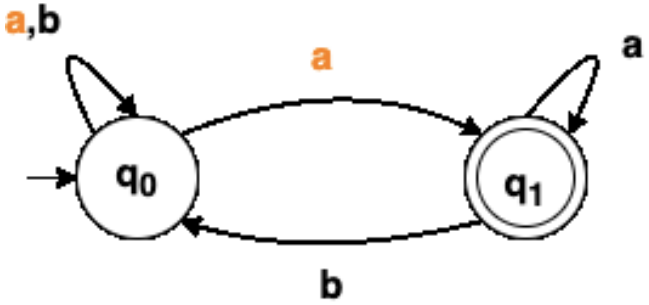
Automates finis déterministes

Exemple :Σ = {a,b}

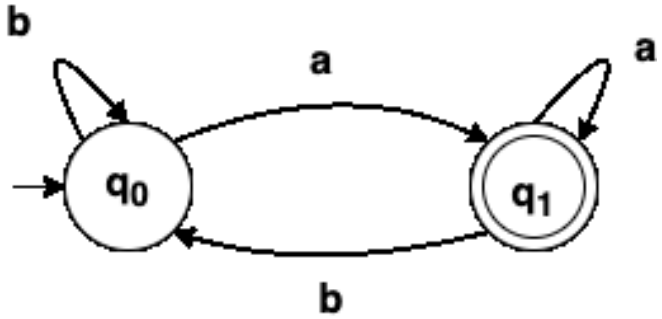
- Automate non complet sur Σ car $\delta(q_0, b) = \phi$



- Automate ambigu sur {a, b} car $\delta(q_0, a) = \{q_0, q1\}$



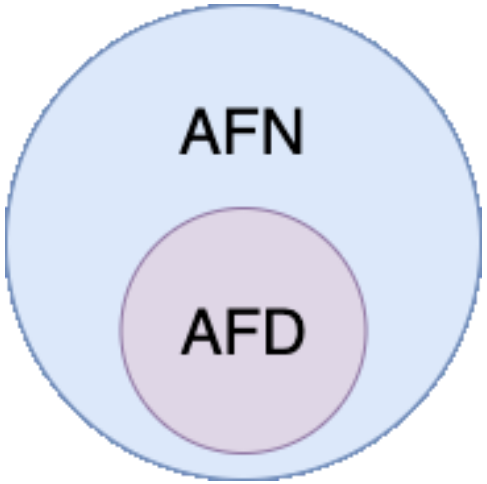
- Automate fini déterministe sur Σ car il est complet et non ambigu:
  δ(q0, a) = {q1}, δ(q0, b) = {q0}
  δ(q1, a) = {q1}, δ(q1, b) = {q0}



Automates finis déterministes

Remarques :

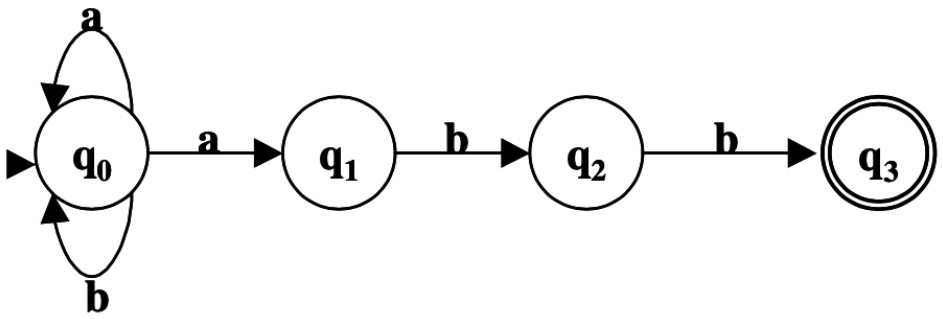
- Dans un automate fini non déterministe (AFN) il peut y avoir le choix entre plusieurs chemins lors de la lecture d'un mot.
  - Pour qu'un mot soit accepté, il suffit que ses lettres étiquettent un chemin d'un état initial à un état final
- Un AFD est un AFN particulier.

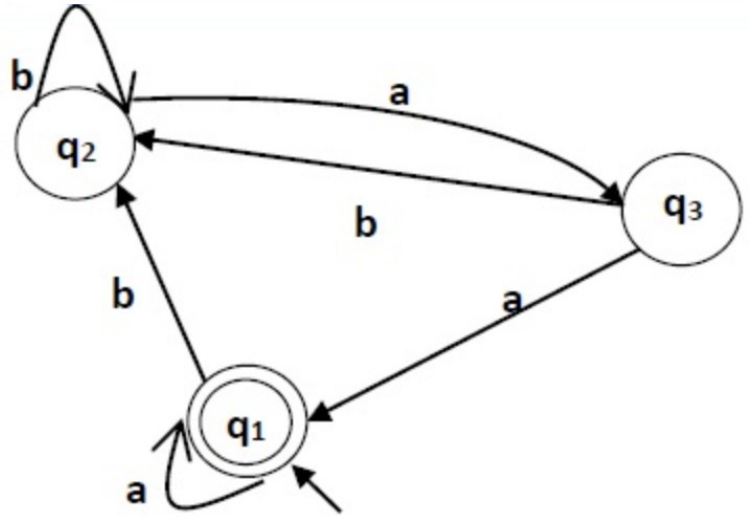


Automates finis déterministes

Exercice :

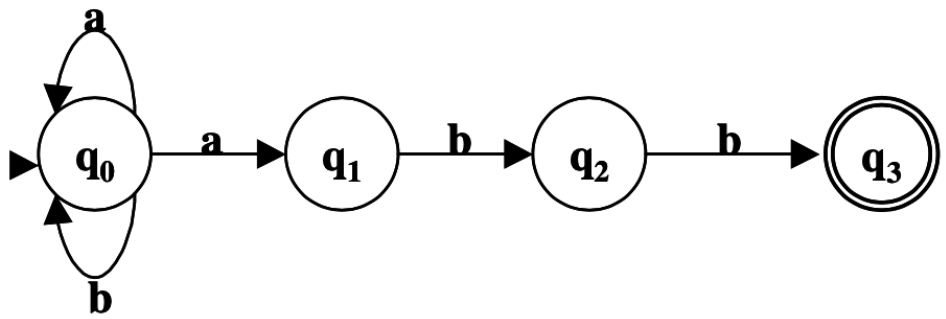
Les automates suivants sont-ils déterministes?





Automates finis déterministes

Correction :

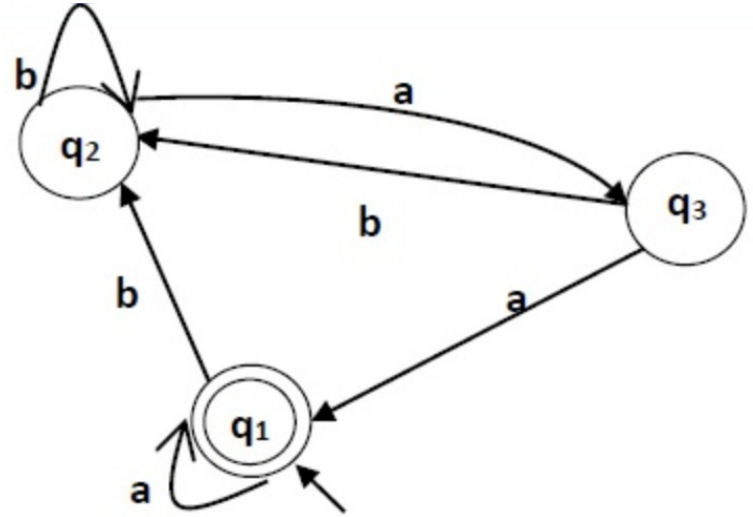


	a	b
q_0	q_0, q_1	q_0
q_1	-	q_2
q_2	-	q_3
q_3	-	-

Automate fini non déterministe

Automates finis déterministes

Correction :



	a	b
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_1	q_2

Automate fini déterministe

Automates finis déterministes

Exercice :

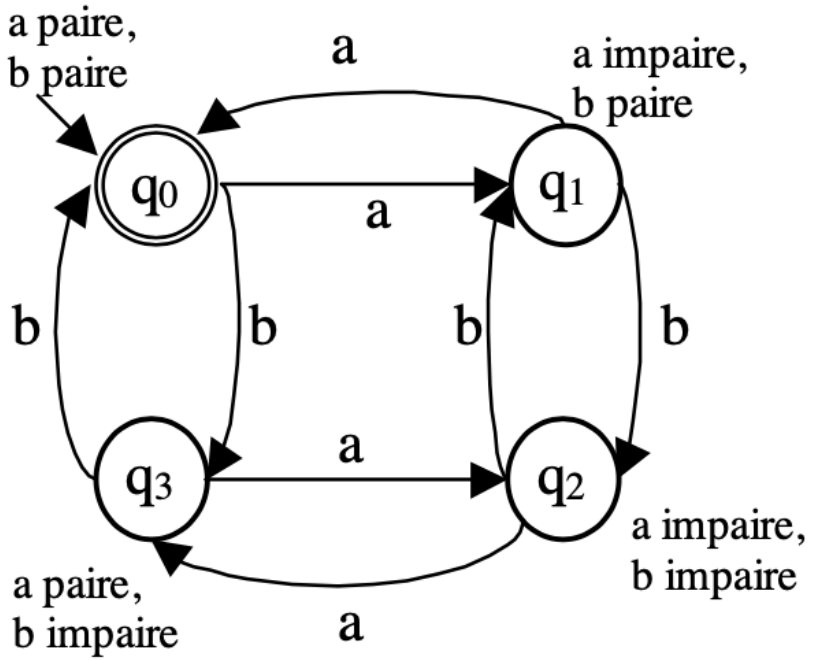
Proposer pour chacun des langages suivant un automate fini déterministe :

- $L_1 = \{\omega \in \{a,b\}^*, \omega$ contient un nombre pair de a et un nombre pair de b$\}$
- $L_2 = \{\omega \in \{a,b,c\}^*, \omega$ contient $(3k+1)a, k \geq 0\}$
- $L_3 = \{\omega \in \{0,1,2,3,4,5,6,7,8,9\}^*, \omega$ divisible par 3$\}$

Automates finis déterministes

Correction :

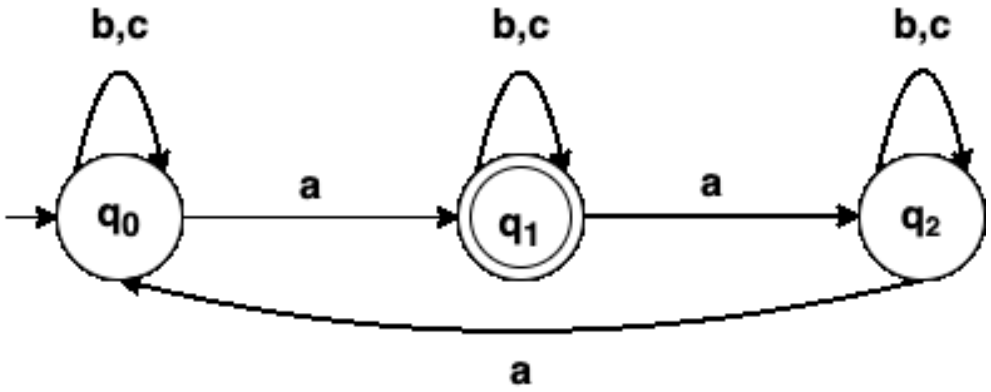
- L_1 = {ω ∈ {a,b}*, ω contient un nombre pair de a et un nombre pair de b}



Automates finis déterministes

Correction :

- $L_2 = \{\omega \in \{a,b,c\}^*, \omega \text{ contient } (3k+1)a, k \geq 0\}$



Automates finis déterministes

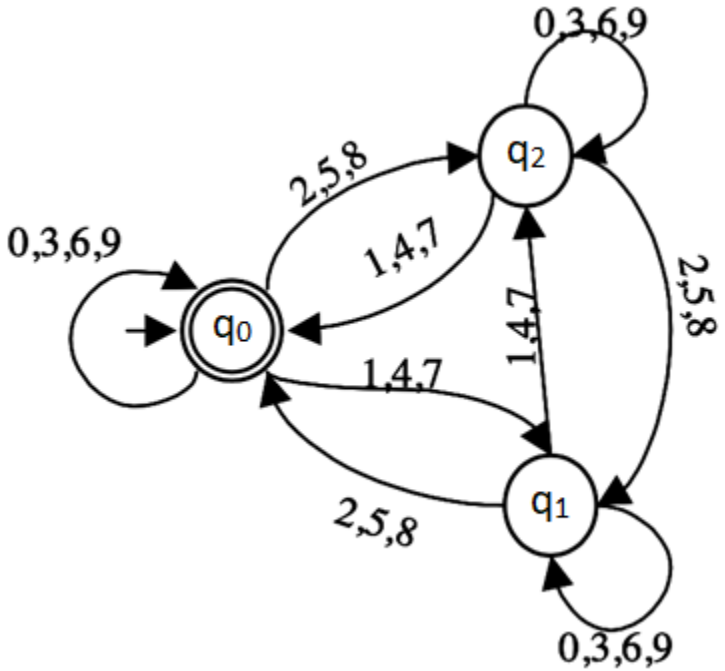
Correction :

- L_3 = {ω ∈ {0,1,2,3,4,5,6,7,8,9}*, ω divisible par 3}

➥ La somme des chiffres est divisible par 3

On part de l'idée :

- chiffre mod 3 = 0 :{0,3,6,9}
- chiffre mod 3 = 1 :{1,4,7}
- chiffre mod 3 = 2 :{2,5,8}



Rendre déterministe un AFN

Théorème:

Pour chaque automate fini non déterministe (AFN) correspond un automate fini déterministe (AFD) qui lui est équivalent.

Autrement dit:

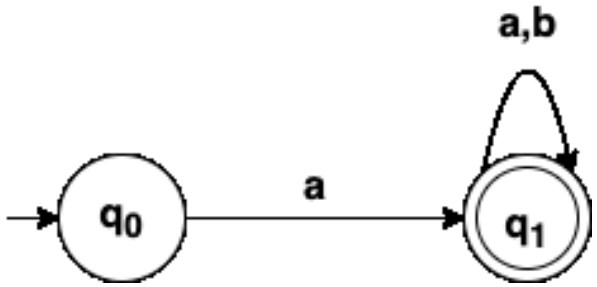
Si un langage est reconnu par un automate, alors il est également reconnu par un automate fini déterministe

- AFN (non complet)→AFD
- AFN (ambigu)→AFD

Rendre déterministe un AFN

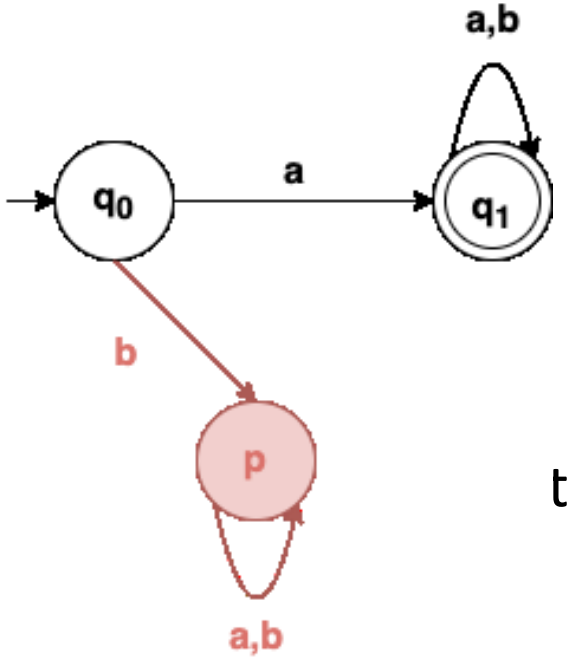
Automate fini non complet :

Si l'automate fini est non déterministe par ce qu'il est non complet, alors pour le rendre déterministe, il suffit d'ajouter un état 'puit' et ajouter toutes les transitions manquantes vers cet état.



Non complet sur {a, b}

car δ(q0, b) = ϕ



t sur {a, b}

Rendre déterministe un AFN

Automate fini ambigu :

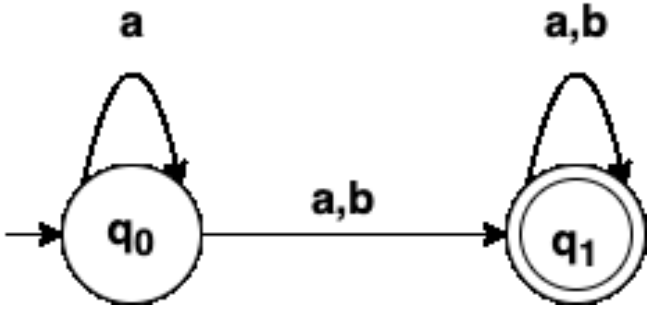
Si l'automate fini est non déterministe par ce qu'il est ambigu alors on doit construire un automate en groupant les états visités par un même symbole à partir d'un état, ces états groupés forment les nouveaux états de l'automate déterministe. La construction de l'automate déterministe suit les étapes suivantes:

- **Etape 1:** Définir les nouveaux groupes d'états
- **Etape 2:** Renommer les nouveaux groupes et définir les états finaux. Un nouvel état (groupe d'état) est un état final ssi il contient un ancien état final
- **Etape 3:** Construire l'automate déterministe

Rendre déterministe un AFN

Automate fini ambigu :

Etape 1: Définir les nouveaux groupes d'états

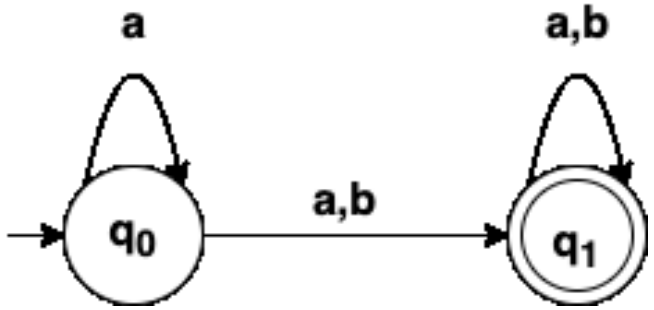


	a	b
{q0}	{q0,q1}	{q1}
{q0,q1}	{q0,q1}	{q1}
{q1}	{q1}	{q1}

Rendre déterministe un AFN

Automate fini ambigu :

Etape 2: Renommer les états et définir les états finaux

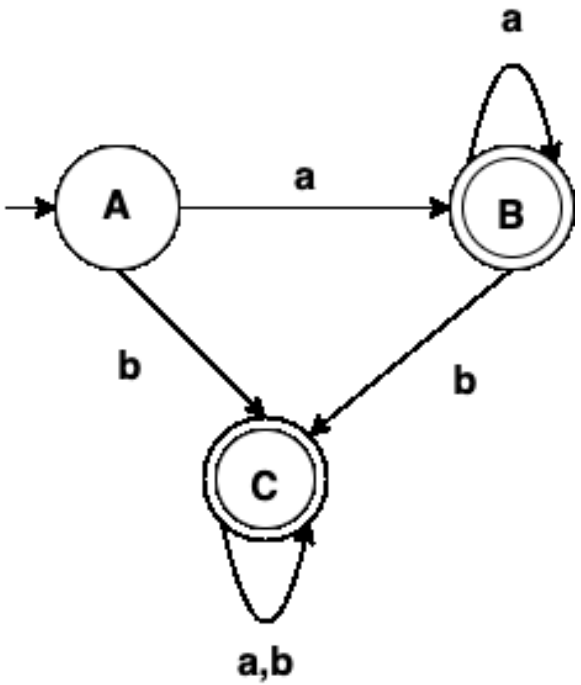


	a	b
{q0} **A**	{q0,q1} **B**	{q1} **C**
{q0,q1} **B**	{q0,q1} **B**	{q1} **C**
{q1} **C**	{q1} **C**	{q1} **C**

Rendre déterministe un AFN

Automate fini ambigu :

Etape 3: Construire l'automate déterministe

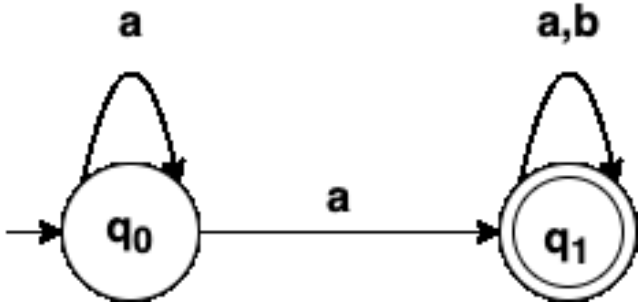


	a	b
{q0} A	{q0,q1} B	{q1} C
{q0,q1} B	{q0,q1} B	{q1} C
{q1} C	{q1} C	{q1} C

Rendre déterministe un AFN

Exercice :

Soit l'automate suivant, construire un automate fini déterministe qui lui correspond



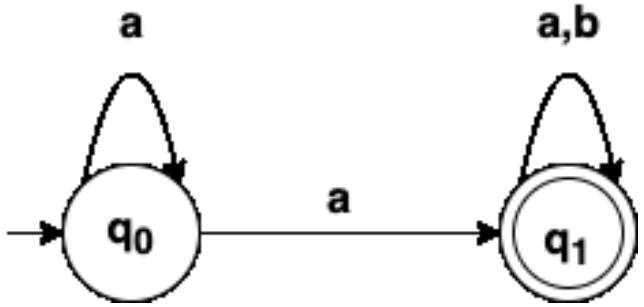
	a	b
{q0}	{q0,q1}	Ø
{q1}	{q1}	{q1}
Ø	Ø	Ø

Rendre déterministe un AFN

Correction :

Etape 1: Définir les nouveaux groupes d'états

Etape 2: Renommer les états et définir les états finaux

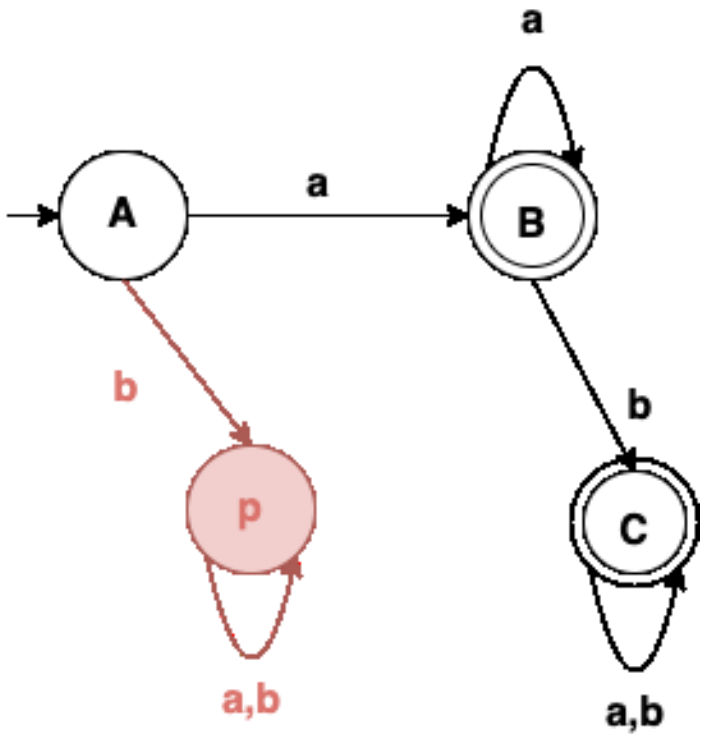


	a	b
{q0} **A**	{q0,q1} **B**	Ø **P**
{q0,q1} **B**	{q0,q1} **B**	{q1} **C**
{q1} **C**	{q1} **C**	{q1} **C**
Ø **P**	Ø **P**	Ø **P**

Rendre déterministe un AFN

Correction :

Etape 3: Construire l'automate déterministe

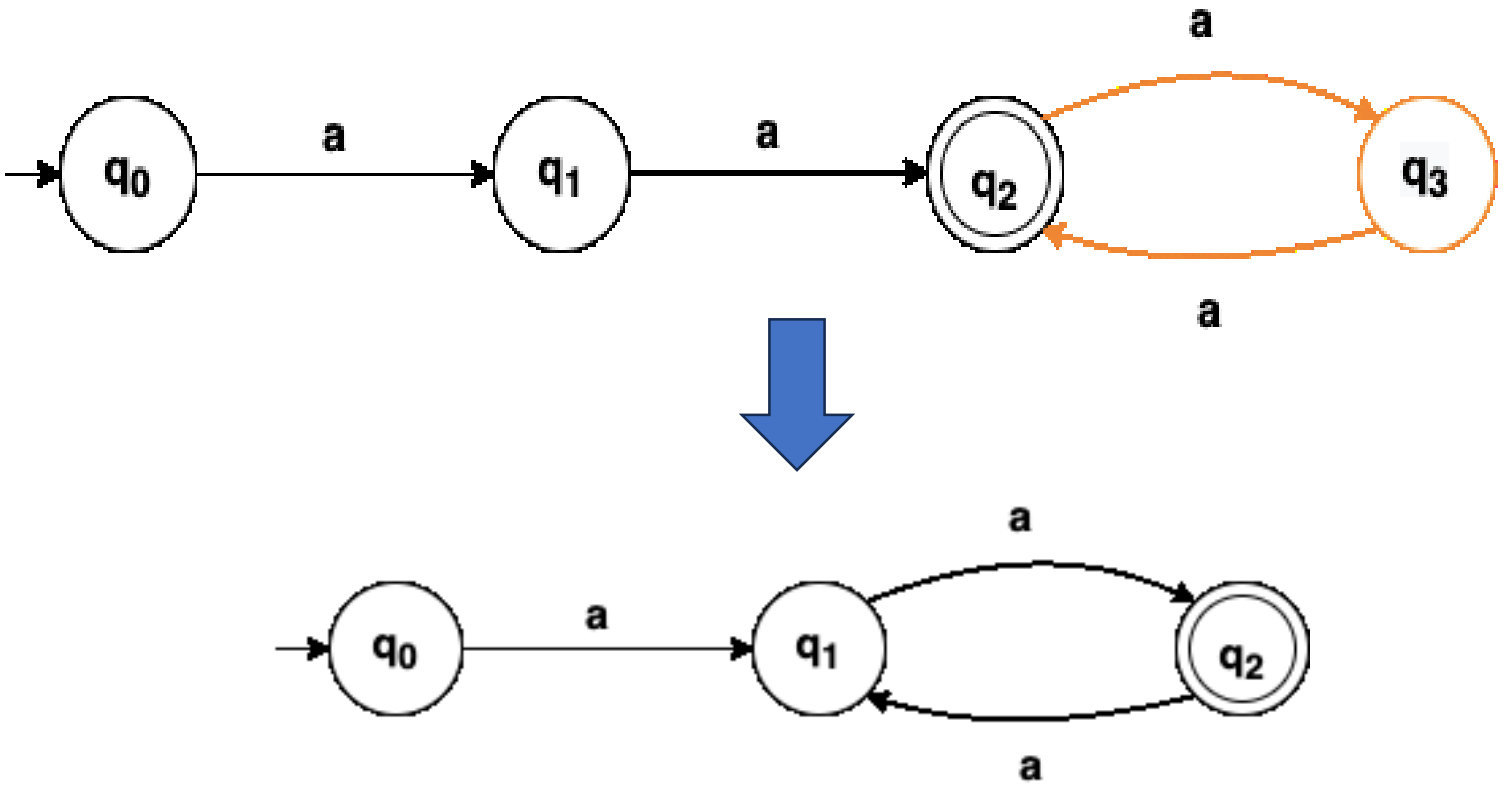


	a	b
{q0} A	{q0,q1} B	Ø P
{q0,q1} B	{q0,q1} B	{q1} C
{q1} C	{q1} C	{q1} C
Ø P	Ø P	Ø P

Minimisation d'AFD

$L = \{\omega \in \{a\}^* / \ |\omega| = 2k, k > 0\}$

aa(aa)*



Minimisation d'AFD

Soit A un automate fini déterministe A = (Q, Σ, δ, q0, F)

Les étapes suivantes nous permettent d'obtenir un automate minimal en nombre d'états :

1) Construire une partition initiale P0 composée de deux groupes: Groupe des états finaux et Groupe des états non finaux

2) Répéter jusqu'à Pi+1=Pi / i≥0

- Pour obtenir Pi+1, partitionner chaque groupe de Pi en mettant ensemble des états p et q si pour chaque symbole s de l'alphabet, p et q transitent vers des états d'un même groupe d'états.
- Construire les nouveaux groupes

3) Associer à chaque groupe un nom

4) Construire les transitions des groupes en utilisant les transitions des états des groupes

5) Un groupe qui contient un état final est un état final dans l'automate minimal

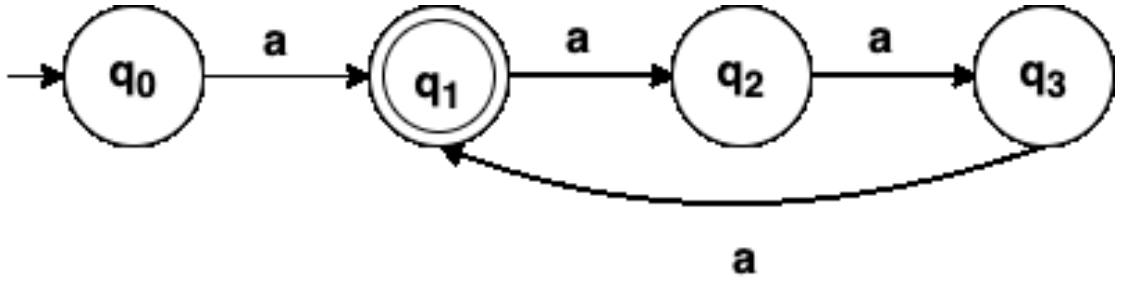
Minimisation d'AFD

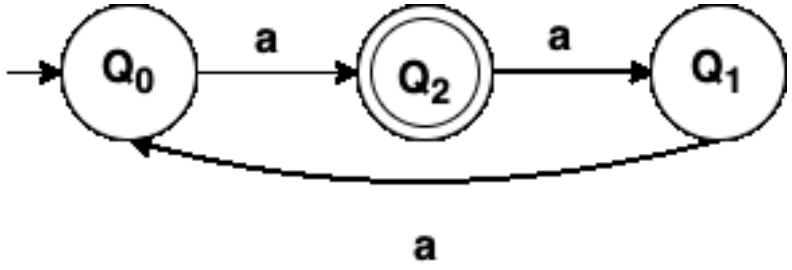
Exemple :

Soit l'automate A suivant. Construire un automate minimal qui accepte le même langage (minimiser le nombre d'états de l'automate A)

	G1	G2
P0	(q0,q2,q3)	(q1)

	G1	G2	G3
P1	(q0,q3)	(q2)	(q1)
P2	(q0,q3)	(q2)	(q1)
	Q0	Q1	Q2

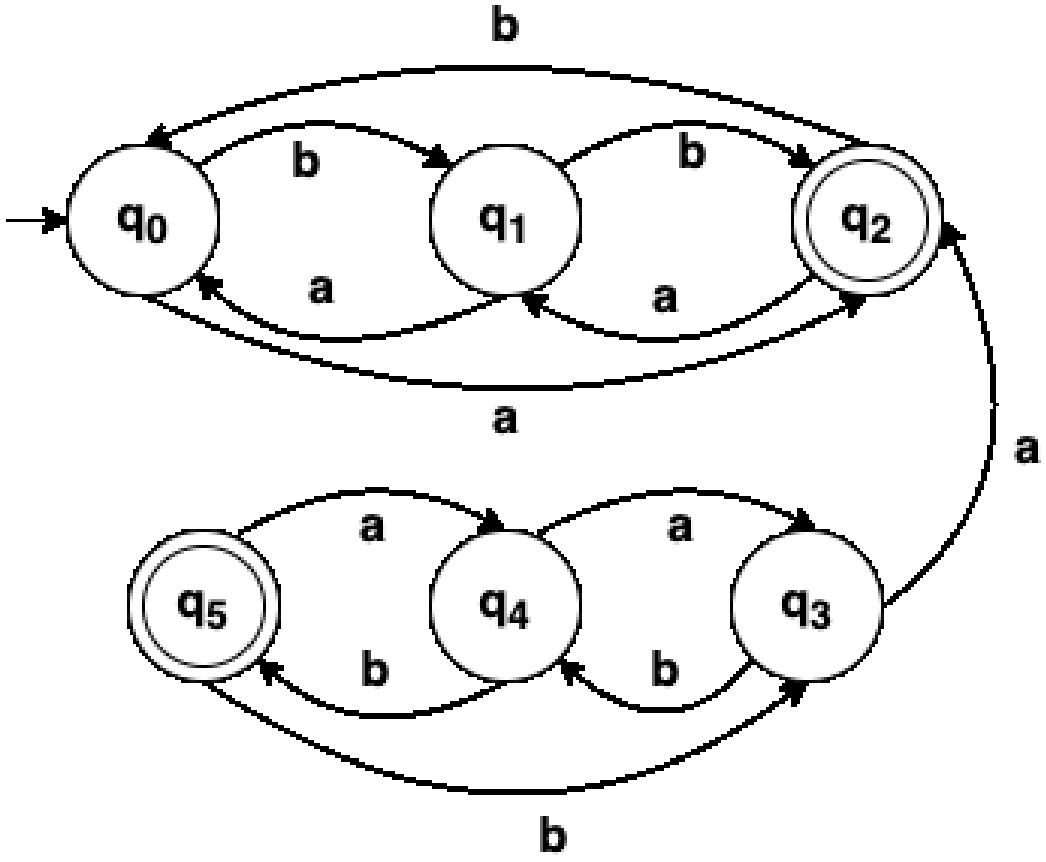




Minimisation d'AFD

Exercice :

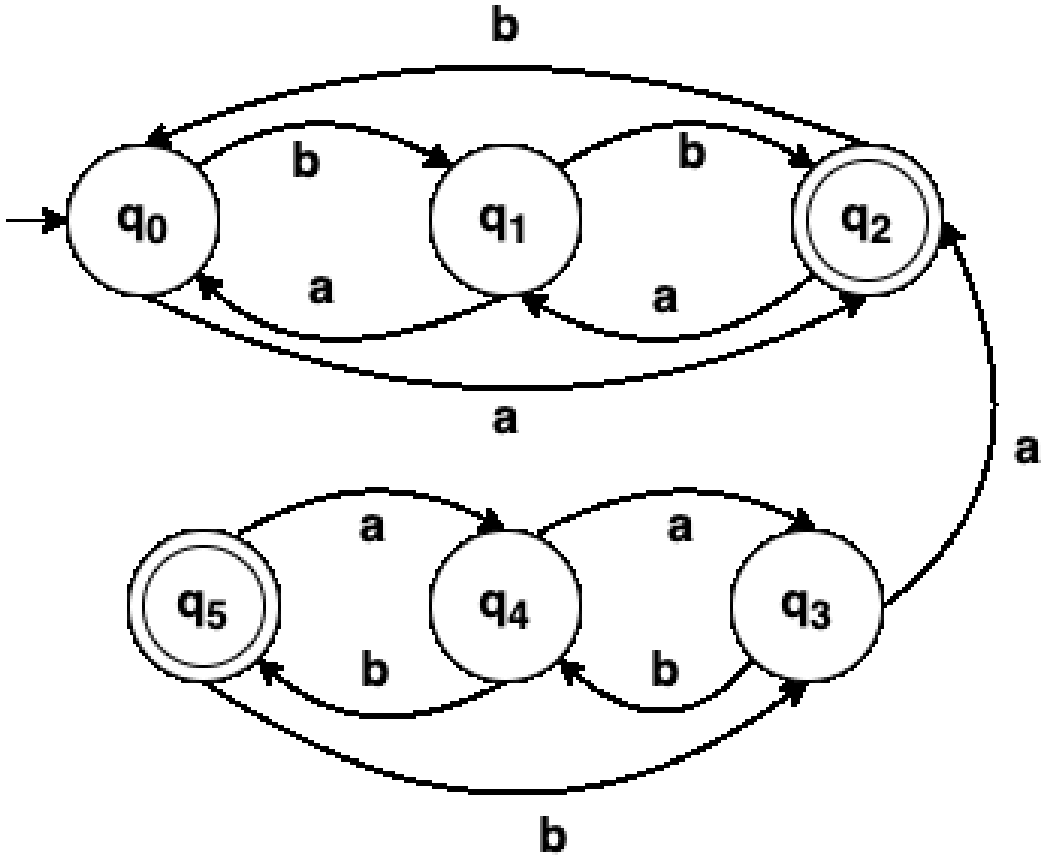
Minimiser l’automate suivant :



Minimisation d'AFD

Correction :

		q0	q1	q2	q3	q4	q5
P0	→	G1	G1	G2	G1	G1	G2
	a	G2	G1	G1	G2	G1	G1
	b	G1	G2	G1	G1	G2	G1
P1	→	G1	G2	G3	G1	G2	G3
	a	G3	G1	G2	G3	G1	G2
	b	G2	G3	G1	G2	G3	G1
P2	→	G1	G2	G3	G1	G2	G3
		A	B	C	A	B	C

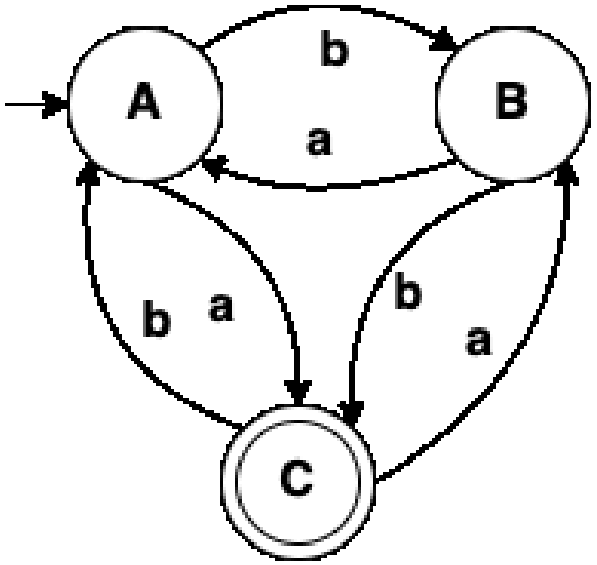


Minimisation d'AFD

Correction :

		q0	q1	q2	q3	q4	q5
P0	→	G1	G1	G2	G1	G1	G2
	a	G2	G1	G1	G2	G1	G1
	b	G1	G2	G1	G1	G2	G1
P1	→	G1	G2	G3	G1	G2	G3
	a	G3	G1	G2	G3	G1	G2
	b	G2	G3	G1	G2	G3	G1
P2	→	G1	G2	G3	G1	G2	G3
		A	B	C	A	B	C

	a	b
A	C	B
B	A	C
C	B	A



Langage accepté par un automate fini

Soit $A = (Q, \Sigma, \delta, q_0, F)$ un automate fini.

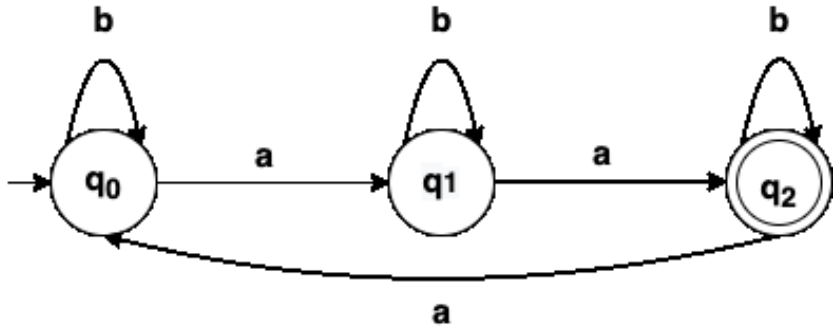
Le langage accepté (ou reconnu par A) est noté L(A) tel que

$$L(A)=\{\omega / (q_0,\omega) \vdash^* (q_f,\varepsilon) , q_f \in F\}$$

ω est accepté par A ssi $\omega \in L(A)$

Exemple :

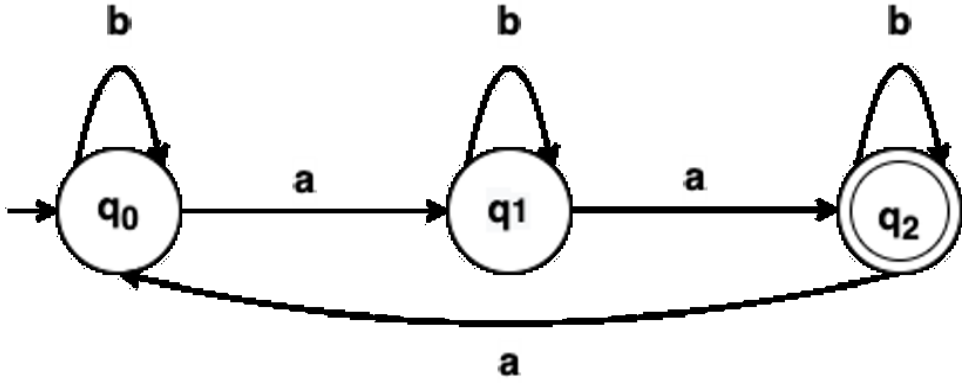
Soit l'automate fini A suivant :



Montrer que $aa \in L(A)$ et que $bba \notin L(A)$

Langage accepté par un automate fini

Exemple :



On a :

aa ∈ L(A)

Car $(q_0, aa) \overset{*}{\vdash} (q_2, \varepsilon)$ en effet $(q_0, aa) \vdash (q_1, a) \vdash (q_2, \varepsilon)$ $(k = 2)$

bba ∉ L(A)

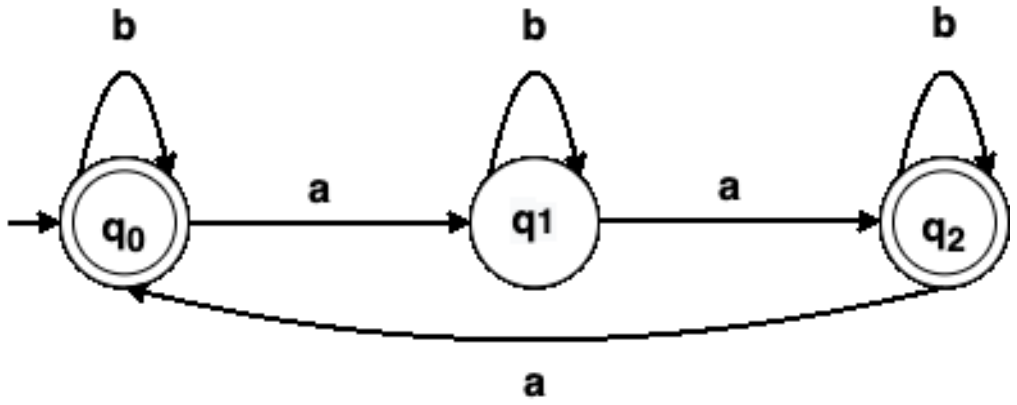
Car $\nexists k$ / $(q_0,bba) \overset{k}{\vdash} (q_2,\varepsilon)$

En effet, $(q_0, bba) \vdash (q_0, ba) \vdash (q_0, a) \vdash (q_1, \varepsilon)$

Langage accepté par un automate fini

Exercice :

On considère l'automate A suivant :

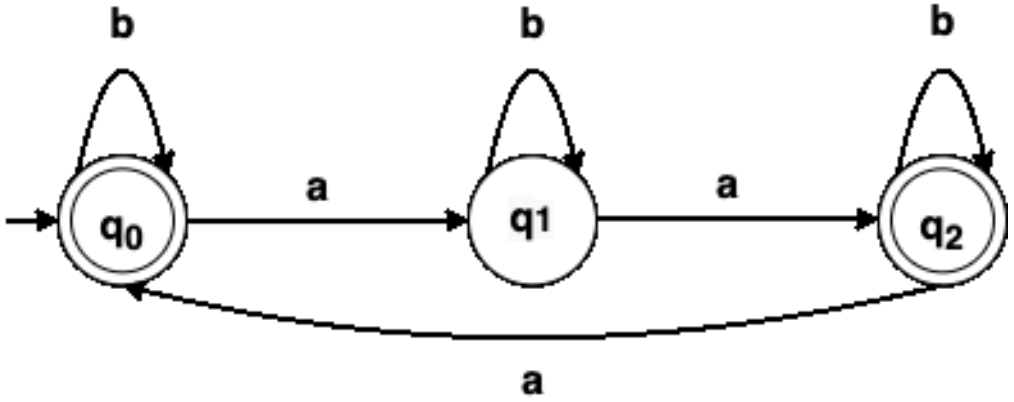


1) Quel est le langage reconnu par A

2) Est-ce que le mot vide ε est reconnu par A

Langage accepté par un automate fini

Correction :



1) $L(A)=\{\omega\in\{a,b\}^*/|\omega|_a=3k,k\geq 0\}\cup\{\omega\in\{a,b\}^*/|\omega|_a=2+3k,k\geq 0\}$

2) $\varepsilon\in L(A)$ parce que: $q_0\in F$
   (q_0, ε) est une configuration initiale et finale.

Langage accepté par un automate fini

Exercice :

Soit l'automate A=(Q, Σ, δ, s, F) avec :

- Σ={a, b}
- Q={q0, q1}
- s=q0
- F={q0}

δ	**a**	**b**
q0	q0	q1
q1	q1	q0

Les mots **ababa** et **aba** sont-ils acceptés par A ?

Langage accepté par un automate fini

Exercice :

(q0,ababa) ⊢ (q0,baba)

⊢ (q1,aba)

⊢ (q1,ba)

⊢ (q0,a)

⊢ (q0,ε)

δ	**a**	**b**
q0	q0	q1
q1	q1	q0

q0 est un état d'acceptation ➠ ababa est accepté par A

(q0, aba) ⊢ (q0,ba) ⊢ (q1, a) ⊢ (q1, ε)

q1 ∉ F ➠ aba n'est pas accepté par A

Langage accepté par un automate fini

Lemme d'Arden :

Pour deux langages A et B de Σ*, Les équations L= ALUB et L = LAUB admettent respectivement comme solution minimale A*B et BA*.

Cette solution est unique si ε∉A.

Exemple :

L'équation L= ALUB, avec A=x*y et B=x* a l'unique solution (x*y)*x*=(xUy)*

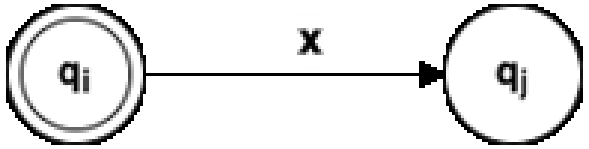
Langage accepté par un automate fini

Lemme d'Arden :

Soit A un automate fini sur un alphabet Σ Avec x ∈ Σ

On dénote par Li, le langage reconnu par l'automate A en considérant que qi est l'état initial. Ainsi, L_0 est le langage reconnu par A

ωj ∈Lj⇔∃ωi ∈Li/ωi=xωj en conséquence Li=xLj



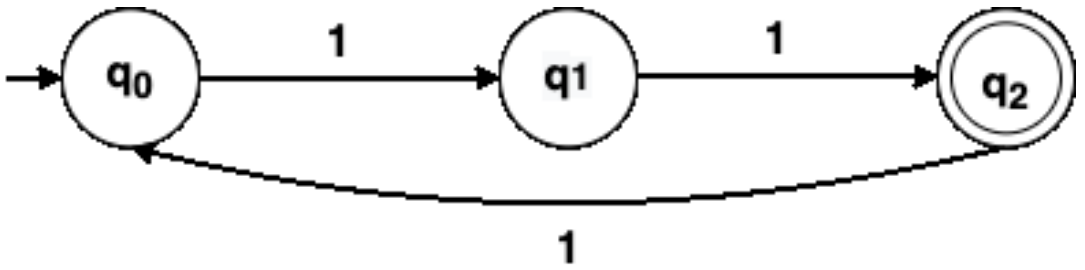
Si qi est un état final alors Li=xLj∪{ε} parce que si qi est un état initial et final alors le langage Li contient le mot ε

Langage accepté par un automate fini

Exemple :

Σ = {0,1}

Soit A un automate fini sur Σ. Chercher le langage L0 reconnu par cet automate.



L0 = {1} L1 (1)

L1 = {1} L2 (2)

L2 = {1} L0 ∪ {ε} (3)

(1) et (2) ⇒ L0= {1}{1}L2={11}L2 (4)

(3) et (4) ⇒ L0={11}({1} L0 ∪ {ε}) = {11}{1}L0 ∪ {11}{ε} = {111}L0 ∪ {11}

L0= (111)*11 (d'après le lemme d'Arden)

Langage accepté par un automate fini

Exercice :

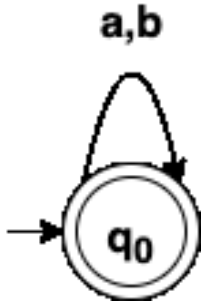
Σ = {a,b}

1. Construire un automate qui accepte le langage (a∪b)* et vérifier que l'automate construit reconnaît le langage (a∪b)*
2. Construire un automate qui accepte le langage a*b* et vérifier que l'automate construit reconnaît le langage a*b*

Langage accepté par un automate fini

Correction:

Σ = {a,b}



L0 = {a} L0 ∪ {b}L0 ∪ {ε} (1)

(1) ⇒L0={a∪b}L0∪{ε}

⇒ L0 = {a∪b}*{ε} (d'après le lemme d'Arden)

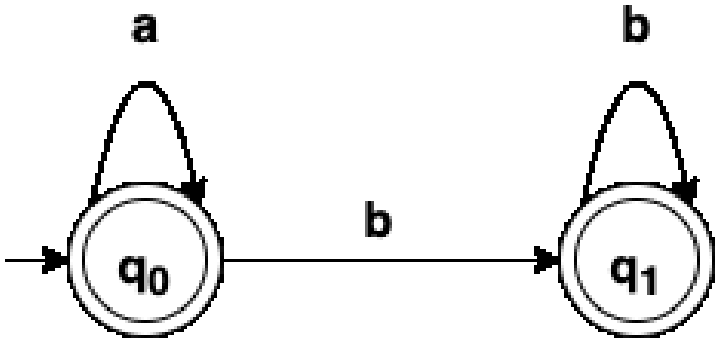
⇒ L0 = {a∪b}* = (a∪b)*

Donc l'automate reconnaît le langage (a∪b)*

Langage accepté par un automate fini

Correction:

Σ = {a,b}



L0 = {a} L0 ∪ {b}L1 ∪ {ε} (1)
L1 = {b} L1 ∪ {ε} (2)

(2) ⇒ L1 = {b}*{ε} (d'après le lemme d'Arden) ⇒ L1 = b* (3)

(1) et (3) ⇒ L0={a}L0 ∪ {b}b* ∪ {ε}

⇒ L0={a}L0 ∪ bb* ∪ {ε}

⇒ L0={a}L0 ∪ b* car bb*∪ {ε}=b*

⇒ L0 = {a}*b* = a*b* (d'après le lemme d'Arden)

L(A) = L0 = a*b* donc l'automate A reconnaît le langage a*b*

Automates avec ε-transitions

Théorème :

Tout langage régulier est reconnaissable par un AFN avec ε-transitions.

Construction de Thompson :

Donnée : une expression régulière r sur un alphabet Σ

Résultat : un AFN qui reconnaît L(r)

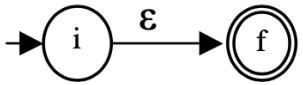
Méthode : on décompose d'abord r en ses sous expressions, puis, en utilisant les règles (1) et (2) pour construire des AFN pour chacun des symboles de base de r (soit ε soit les symboles de l'alphabet).

Ensuite, on combine régulièrement ces AFN en utilisant la règle (3), jusqu'à obtenir l'AFN pour l'expression régulière complète.

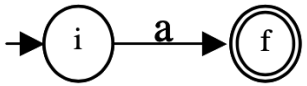
Automates avec ε-transitions

Règles de construction :

1) Pour ε, construire l'AFN :

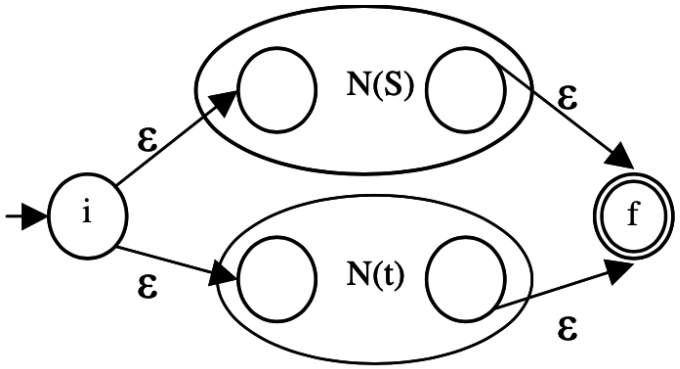


2) Pour a∈Σ, construire l'AFN :



3) Supposons que N(s) et N (t) sont les AFN pour les expressions régulières s et t.

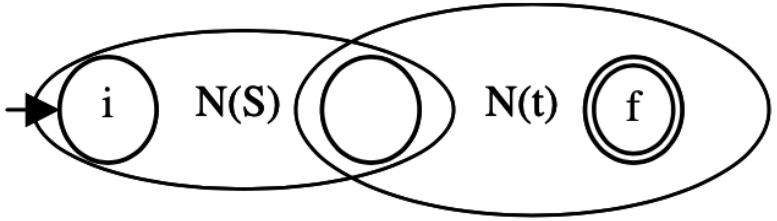
a) pour l'expression régulière sUt, construire l'AFN composé suivant: N (sUt)



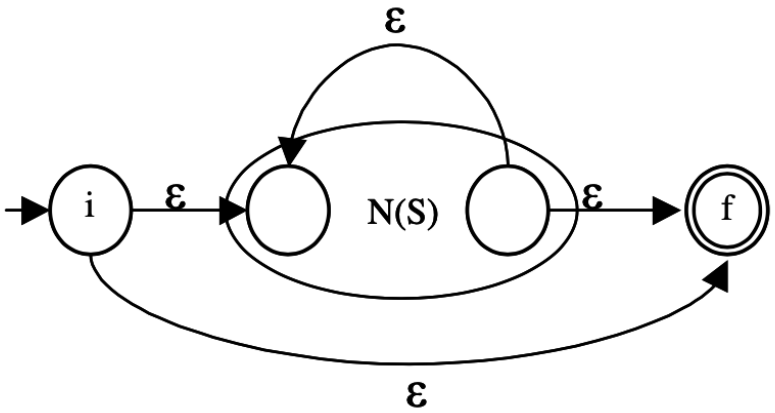
Automates avec ε-transitions

Règles de construction :

b) pour l'expression régulière st , construire l'AFN composé N(st) suivant:



c) pour l'expression régulière s*, construire l'AFN composé N (s*) suivant:



Automates avec ε-transitions

Remarques :

- Chaque fois qu'on construit un nouvel état, on lui donne un nom distinct. Ainsi il ne peut y avoir deux états dans deux sous automates qui aient le même nom.
- Si le même symbole apparaît plusieurs fois dans r, on crée, pour chaque instance de ce symbole, un AFN séparé avec ses propres états.

Exemple:

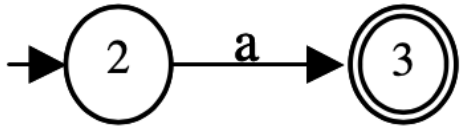
soit r = (aUb)*abb

Automates avec ε-transitions

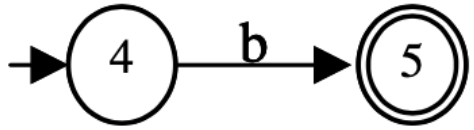
Exemple:

r = (a∪b)*abb

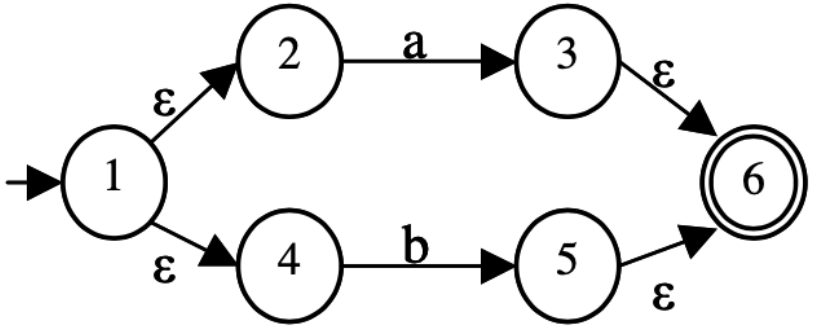
Pour r1 on construit le AFN :



Pour r2 on construit le AFN :



On peut maintenant combiner N(r1) et N(r2) on utilisant la règle de l'union pour obtenir N (r1 ∪ r2) :

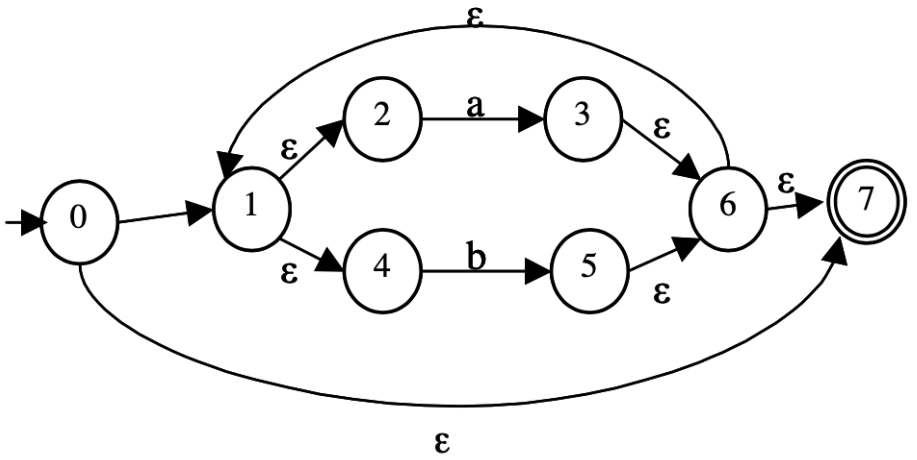


Automates avec ε-transitions

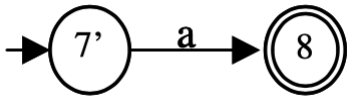
Exemple:

r = (aUb)*abb

Le AFN pour r5=(r4)* est :

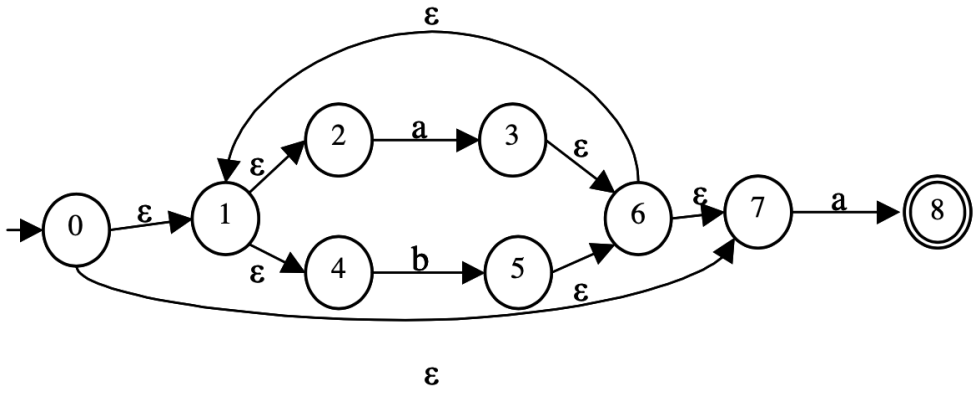


Le NFA pour r6=a est :



Pour r7=r5.r6 on utilise la règle de la concaténation pour obtenir N (r5. r6) :

on fusionne les états 7 et 7‘

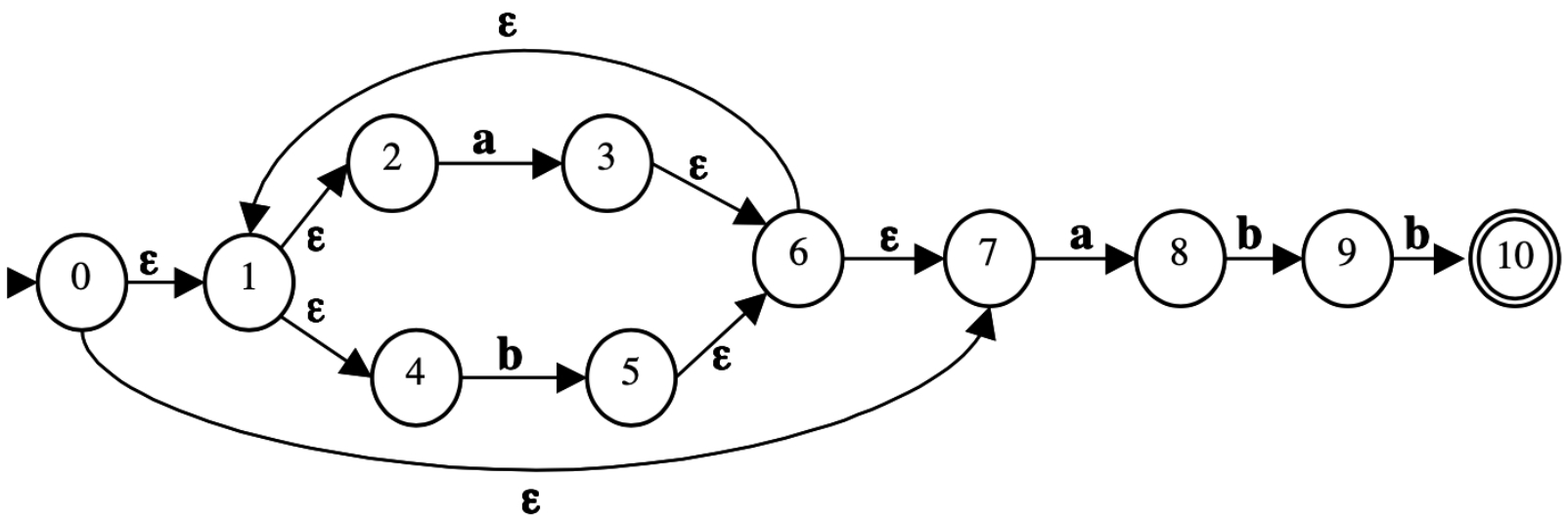


Automates avec ε-transitions

Exemple:

r = (a∪b)*(abb)

En continuant ainsi on obtient l'AFN pour r = r11 = (a∪b)*abb



Automates avec ε-transitions

Algorithme de transformation AFN → AFD :

ε-fermeture (q) : Ensemble des états de l'AFN accessibles depuis un état q de l'AFN par des ε-transitions uniquement.

ε-fermeture (T) : Ensemble des états de l'AFN accessibles depuis un état $q \in T$

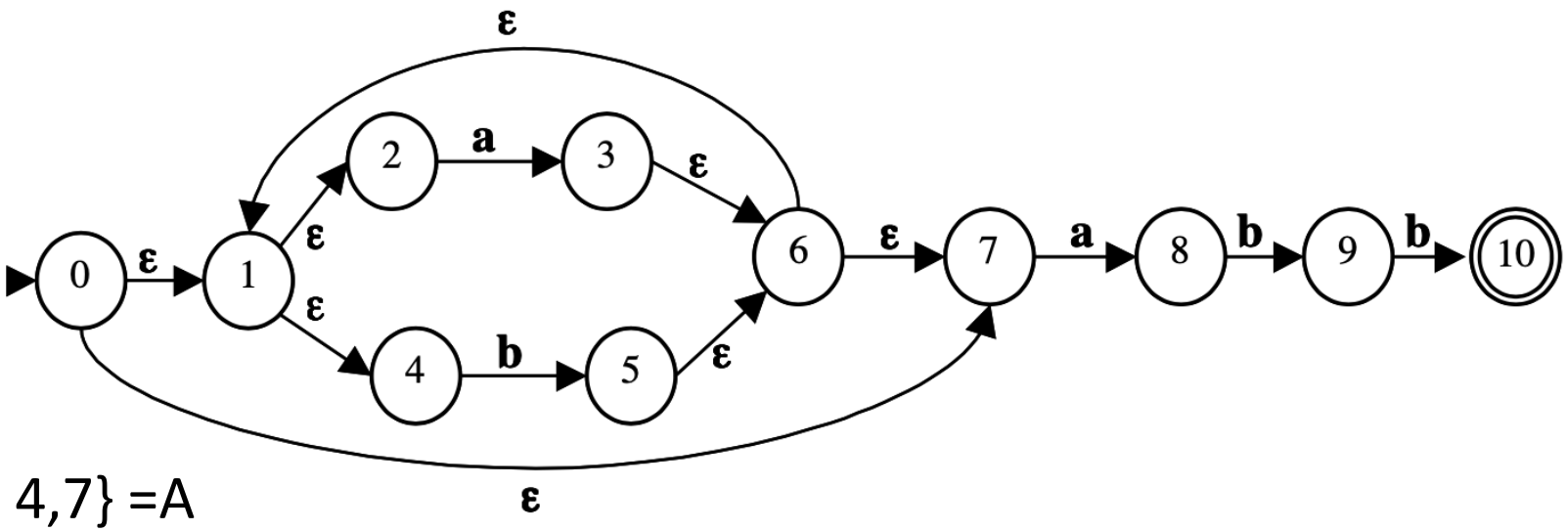
Transiter (T,s) : Ensemble des états de l'AFN vers lequel il existe une transition sur le symbole s à partir d'un état $q \in T$.

Automates avec ε-transitions

Exemple:

Σ={a,b}

r = (a∪b)*abb



Etat de départ:

ε-fermeture (0)= {0, 1, 2, 4,7} =A

Transiter(A,a)={3,8}

On calcule ε-fermeture({3,8})={1,2,3,4,6,7,8}=B ⇒ δ(A,a)=B

Transiter(A,b)={5}

On calcule ε-fermeture({5})={1,2,4,5,6,7}=C ⇒ δ(A,b)=C

Automates avec ε-transitions

Exemple:

Nous continuons ce processus avec les ensembles actuellement non marqués B et C. On atteint finalement le point ou tous les ensembles qui sont des états du AFD sont marqués.

Les cinq différents ensembles que nous construisons réellement sont :

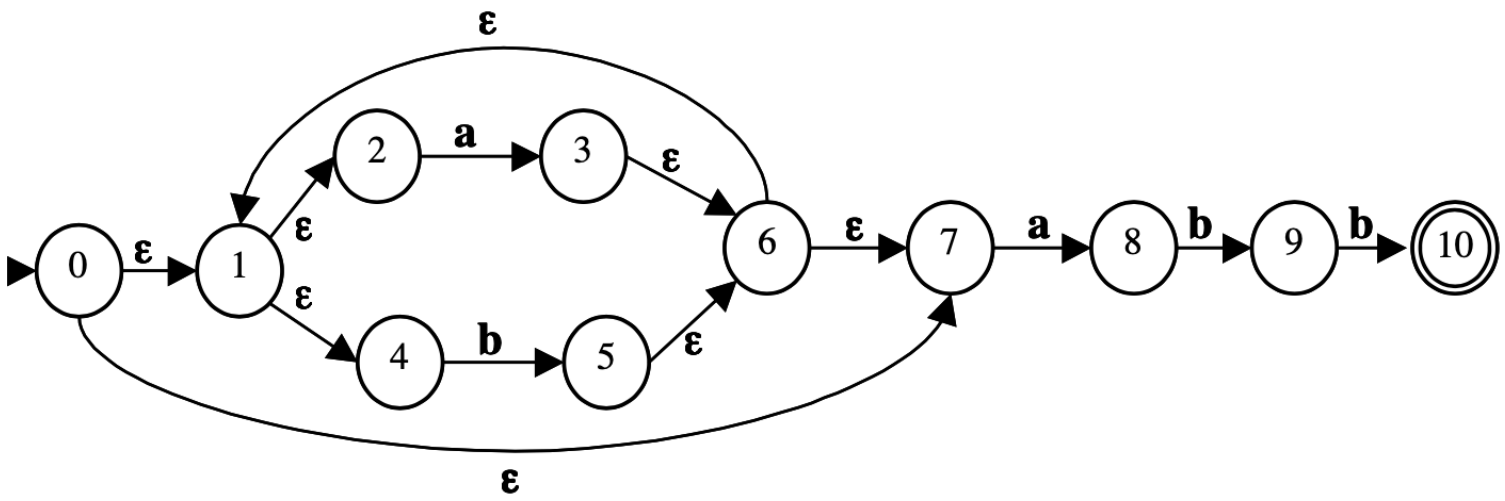
A= {0, 1, 2, 4,7}

B= {1, 2, 3, 4, 6, 7,8}

C= {1, 2, 4, 5,6 ,7}

D= {1, 2, 4, 5, 6, 7,9}

E= {1, 2, 4, 5, 6, 7,10}

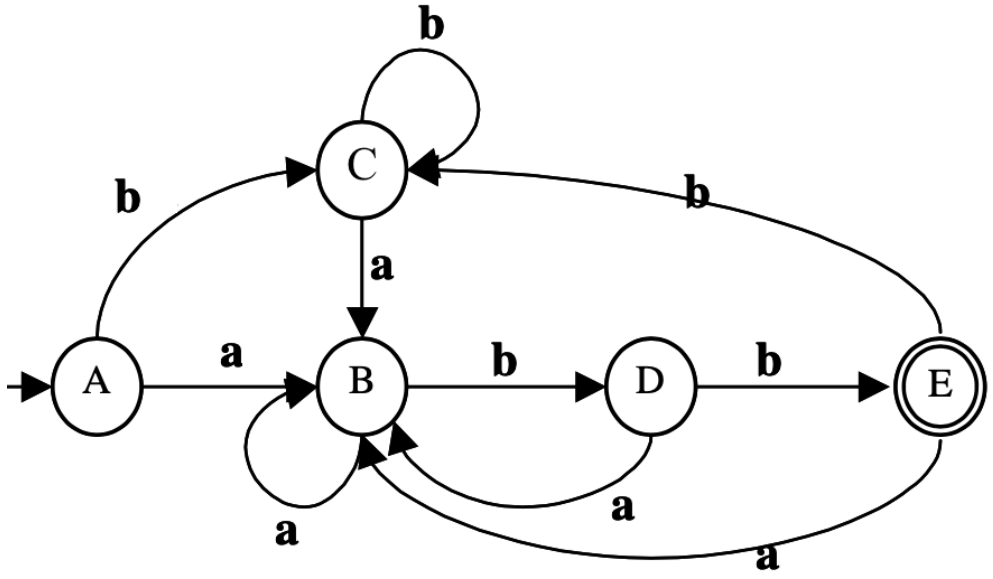


Automates avec ε-transitions

Exemple:

L'état A est l'état de départ et l'état E est l'unique état d'acceptation car il contient un état d'acceptation de l'AFN (10)

δ	a	b
A	B	C
B	B	D
C	B	C
D	B	E
E	B	C



Automates avec ε-transitions

Exercice: (*A faire*)

Σ={0,1}

Soit r = 0*1∪(10)*

1) En utilisant la construction du Thompson, donner l'AFN qui reconnaît L(r)

2) Transformer l'AFN obtenu en un AFD